Object Oriented Programming

Mihai Dascălu

Generics

Why Do You Get a Warning?

```
public class ShowUncheckedWarning {
  public static void main(String[] args) {
    java.util.ArrayList list =
      new java.util.ArrayList();
    list.add("Java Programming");
  }
}
```



To understand the compile warning on this line, you need to learn Generics

Fix the Warning

```
public class ShowUncheckedWarning {

  public static void main(String[] args) {

    java.util.ArrayList<String> list =

      new java.util.ArrayList<String>();

    list.add("Java Programming");

  }

}
```



No compile warning on this line

What is Generics?

- Capability to parameterize types
- Provide the possibility to define a class or a method with generic types that can be substituted using concrete types by the compiler
  - e.g., define a generic stack class that stores the elements of a generic type
  - From this generic class, you may create a stack object for holding strings and a stack object for holding numbers
  - Strings and numbers are concrete types that replace the generic type

Why Generics?

- **Enable errors** to be **detected** at **compilation** rather than at runtime
- A generic class or method permits you to specify allowable types of objects that the class or method may work with
- If you attempt to use the class or method with an incompatible object, the compile error occurs

Generic Type

```
package java.lang;

public interface Comparable {
  public int compareTo(Object o)
}
```

(a) Prior to JDK 1.5

```
package java.lang;

public interface Comparable<T> {
  public int compareTo(T o)
}
```

(b) JDK 1.5

Runtime error

Generic Instantiation

```
Comparable c = new Date();
System.out.println(c.compareTo("red"));
```

(a) Prior to JDK 1.5

```
Comparable<Date> c = new Date();
System.out.println(c.compareTo("red"));
```

(b) JDK 1.5

Improve reliability

Compilation error

Generic ArrayList starting from JDK 1.5

java.util.ArrayList
+ArrayList()
+add(o: Object) : void
+add(index: int, o: Object) : void
+clear(): void
+contains(o: Object): boolean
+get(index: int) : Object
+indexOf(o: Object) : int
+isEmpty(): boolean
+lastIndexOf(o: Object) : int
+remove(o: Object): boolean
+size(): int
+remove(index: int) : boolean
+set(index: int, o: Object) : Object

(a) ArrayList before JDK 1.5

java.util.ArrayList<E>
+ArrayList()
+add(o: E) : void
+add(index: int, o: E) : void
+clear(): void
+contains(o: Object): boolean
+get(index: int) : E
+indexOf(o: Object) : int
+isEmpty(): boolean
+lastIndexOf(o: Object) : int
+remove(o: Object): boolean
+size(): int
+remove(index: int) : boolean
+set(index: int, o: E) : E

(b) ArrayList in JDK 1.5

No Casting is Needed

```
ArrayList<Double> list = new ArrayList<>();

list.add(5.5); // 5.5 is automatically
 converted to new Double(5.5)

list.add(3.0); // 3.0 is automatically
 converted to new Double(3.0)

Double doubleObject = list.get(0); // No
 casting is needed

double d = list.get(1); // Automatically
 converted to double
```

Declaring Generic Classes and Interfaces

GenericStack<E>	
-list: java.util.ArrayList<E>	An array list to store elements.
+GenericStack()	Creates an empty stack.
+getSize(): int	Returns the number of elements in this stack.
+peek(): E	Returns the top element in this stack.
+pop(): E	Returns and removes the top element in this stack.
+push(o: E): E	Adds a new element to the top of this stack.
+isEmpty(): boolean	Returns true if the stack is empty.

GenericStack

Generic Methods

```
public static <E> void print(E[] list) {
   for (int i = 0; i < list.length; i++)
     System.out.print(list[i] + " ");
   System.out.println();
 }

public static void print(Object[] list) {
   for (int i = 0; i < list.length; i++)
     System.out.print(list[i] + " ");
   System.out.println();
 }

class X { public <X> X(X x) { } }
class X { public <X> X X(X x) { } }
```

Raw Type and Backward Compatibility

```
// raw type

ArrayList list = new ArrayList();
```

This is roughly equivalent to

```
ArrayList<Object> list = new ArrayList<Object>();
```

Raw Type is Unsafe

Max.max("Welcome", 23);

```
// Max.java: Find a maximum object
public class Max {
  /** Return the maximum between two objects */
  public static Comparable max(Comparable o1,
    Comparable o2) {
    if (o1.compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

Runtime Error

Safe Version to the Rescue

Max.max("Welcome", 23);

```
// Max1.java: Find a maximum object
public class Max1 {
  /** Return the maximum between two objects */
  public static <E extends Comparable<E>> E
    max(E o1, E o2) {
    if (o1.compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

Compilation Error

Bounded Generic Type

```
public static void main(String[] args ) {
    Rectangle rectangle = new Rectangle(2, 2);
    Circle circle = new Circle(2);
    System.out.println("Same area? " +
equalArea(rectangle, circle));
}

public static <E extends GeometricObject> boolean
       equalArea(E object1, E object2) {
    return object1.getArea() == object2.getArea();
}
```

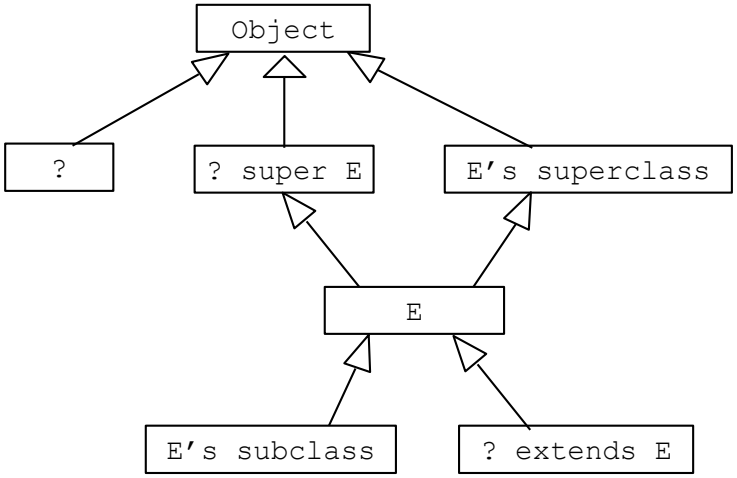
Wildcards

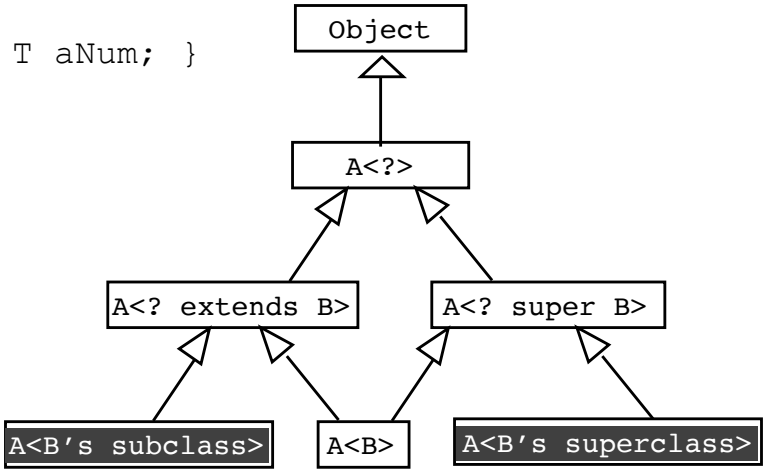
- ? unbounded wildcard
- ? extends T bounded wildcard
- ? super T lower bound wildcard

```
public class NumberHolder<? extends Number> { }

public class NumberHolder<?> { ? aNum; }

public class NumberHolder<T> { T aNum; }
```





WildCardDemo 1-3 X

Restrictions on Generics

- **Restriction 1**: cannot create an instance of a generic type (i.e., new E())
- **Restriction 2**: generic array creation is not allowed (i.e., new E[100])
- **Restriction 3**: a generic type parameter of a class is not allowed in a static context
- **Restriction 4**: exception classes cannot be generic

Erasure and Restrictions on Generics

- Generics are implemented using an approach called ***type erasure***
- The compiler uses the generic type information to compile the code, but erases it afterwards
- So, the generic information is not available at run time
- This approach enables the generic code to be backward-compatible with the legacy code that uses raw types

Use Case 1

- What is the following class converted to after type erasure?

```
public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey(); { return key; }
    public V getValue(); { return value; }

    public void setKey(K key)     { this.key = key; }
    public void setValue(V value) { this.value = value; }
}
```

Use Case 1

- What is the following class converted to after type erasure?

```
public class Pair {
    public Pair(Object key, Object value) {
        this.key = key;
        this.value = value;
    }

    public Object getKey()   { return key; }
    public Object getValue() { return value; }

    public void setKey(Object key)     { this.key = key; }
    public void setValue(Object value) { this.value = value; }

    private Object key;
    private Object value;
}
```

Use Case 2

- What is the following method converted to after type erasure?

```
public static <T extends Comparable<T>>
    int findFirstGreaterThan(T[] at, T elem) {
    // ...
}
```

Use Case 2

- What is the following method converted to after type erasure?

```
public static int findFirstGreaterThan(Comparable[] at,
Comparable elem) {
    // ...
    }
```

Compile Time Checking

- The compiler checks whether generics are used correctly and translates it into the equivalent code for runtime use
- The code in (b) uses the raw type

```
ArrayList<String> list = new ArrayList<String>();
list.add("Oklahoma");
String state = list.get(0);
```

(a)

```
ArrayList list = new ArrayList();
list.add("Oklahoma");
String state = (String)(list.get(0));
```

(b)

Important Facts

- It is important to note that a generic class is shared by all its instances regardless of its actual generic type:

```
GenericStack<String> stack1 = new
  GenericStack<String>();
GenericStack<Integer> stack2 = new
  GenericStack<Integer>();
```

- Although GenericStack<String> and GenericStack<Integer> are two types, but there is only one class GenericStack loaded into the JVM
- Also, be careful that **class parametric types** are not **available** in **static** methods!

Polymorphism

- Polymorphic assignments applies only to the base type, not the generic type parameter:

```
List<Animal> aList = new ArrayList<Animal>(); // yes

List<Animal> aList = new ArrayList<Dog>(); // no
```

- The polymorphic assignment rule applies everywhere an assignment can be made >> the following are NOT allowed:

```
void foo(List<Animal> aList) { }

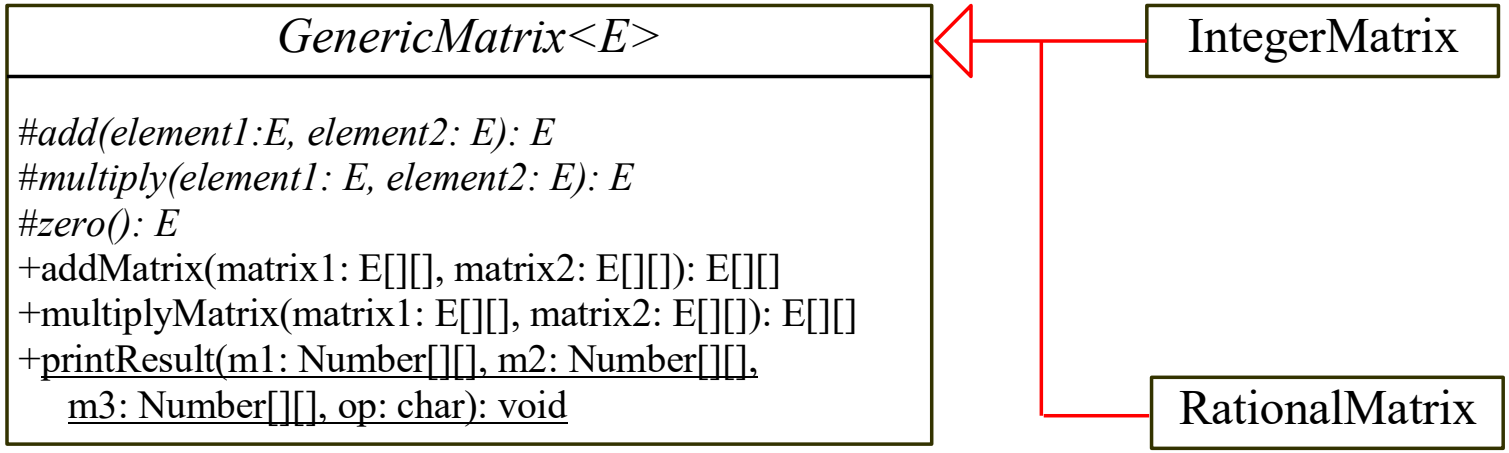
      // cannot take a List<Dog>

List<Animal> bar() { }

      // cannot return a List<Dog>
```

Designing Generic Matrix Classes

- Generic class for matrix arithmetic
- Implement matrix addition and multiplication as common operations for all types of matrices



GenericMatrix, IntegerMatrix, Rational, RationalMatrix

Recap – Why generics?

- Tighter type checks on generic code at compile time
- Support of programming types as parameters
- Elimination of casts
- Enable the implementation of generic algorithms

Use Case 1

- Write a generic method to count the number of elements in a collection that have a specific property (for example, odd integers, prime numbers, palindromes)

```
public final class Algorithm {
    public static <T> int countIf(Collection<T> c, UnaryPredicate<T> p) {
        int count = 0;
        for (T elem : c)
            if (p.test(elem))
                ++count;
        return count;
    }
}
public interface UnaryPredicate<T> {
    public boolean test(T obj);
}
```

Use Case 1

```
class OddPredicate implements UnaryPredicate<Integer> {
    public boolean test(Integer i) { return i % 2 != 0; }
}
public class Test {
    public static void main(String[] args) {
        Collection<Integer> ci = Arrays.asList(1, 2, 3, 4);
        int count = Algorithm.countIf(ci, new OddPredicate());
        System.out.println("Number of odd integers = " + count);
    }
}
```

Use Case 2

- Write a generic method to exchange the positions of two different elements in an array.

```
public final class Algorithm {
    public static <T> void swap(T[] a, int i, int j) {
        ...
    }
}
```

Use Case 2

- Write a generic method to exchange the positions of two different elements in an array.

```
public final class Algorithm {
    public static <T> void swap(T[] a, int i, int j) {
        T temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}
```

Use Case 3

- Write a generic method to find the maximal element in the range [begin, end) of a list.

```
public final class Algorithm {
    public static <T extends Object & Comparable<? super T>>
        T max(List<? extends T> list, int begin, int end) {
        ...
    }
}
```

Use Case 3

- Write a generic method to find the maximal element in the range [begin, end) of a list.

```
public final class Algorithm {
    public static <T extends Object & Comparable<? super T>>
        T max(List<? extends T> list, int begin, int end) {

        T maxElem = list.get(begin);

        for (++begin; begin < end; begin++)
            if (maxElem.compareTo(list.get(begin)) < 0)
                maxElem = list.get(begin);
        return maxElem;
    }
}
```

Algorithm